ESPRESSIF · RISC-V · IOT

ESP32-C6

Power Modes

Active · Modem Sleep · Light Sleep
Deep Sleep · Hibernation · TWT

ESP32-C6 & Power Management

A RISC-V SoC with Wi-Fi 6, BLE 5, 802.15.4 — and a rich set of power modes spanning five orders of magnitude.

RISC-V
32-bit single core

160 MHz
max CPU frequency

Wi-Fi 6
802.11ax + TWT

~5 µA
hibernation draw

~60%
of IoT devices run on batteries

10-100x
battery life gain with sleep modes

>1 year
possible lifetime on coin cell + TWT

Key principle: always use the lowest power mode that meets your latency and connectivity requirements.

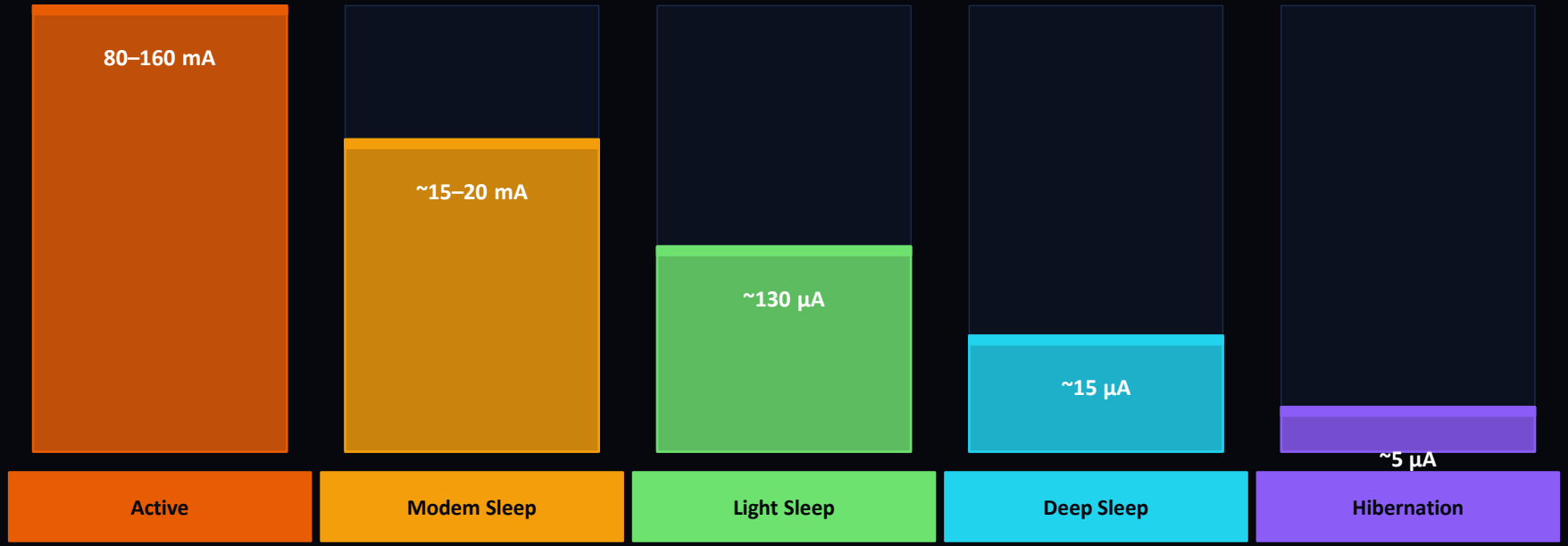
RISC-V Core
Single 32-bit core — deterministic, low-power open ISA

Wi-Fi 6 + TWT
Target Wake Time cuts idle RF power by 10x vs DTIM

HP + LP Memory
High-power SRAM + low-power RTC memory for deep sleep

Power Mode Spectrum

Five modes span more than four orders of magnitude — from full Wi-Fi 6 performance to near-zero standby.



Higher Performance ←

→ Lower Power

ACTIVE MODE

Active Mode

80–160 mA	160 MHz	Full RF	All periph
Current Draw	CPU Frequency	Wi-Fi 6 + BLE	GPIO / I2C etc

When to Use

- Continuous sensor acquisition or computation
- Active Wi-Fi 6 data transmission or reception
- Real-time control loops requiring full 160 MHz
- BLE advertising or active GATT connections

Set CPU Frequency (esp-idf)

```
#include "esp_pm.h"

esp_pm_config_t pm_cfg = {
  .max_freq_mhz = 160,
  .min_freq_mhz = 160,
  .light_sleep_enable = false,
};
ESP_ERROR_CHECK(
  esp_pm_configure(&pm_cfg)
);
```

Tip: DFS (Dynamic Frequency Scaling) can reduce CPU speed during idle bursts, cutting active-mode power by up to 40%.

MODEM SLEEP

Modem Sleep Mode

CPU and memory stay fully operational. The Wi-Fi/BLE radio powers down between DTIM beacon intervals, maintaining the connection.

Component Status

CPU Core (RISC-V)	Active
SRAM / Flash	Retained
Wi-Fi 6 Modem	Periodic (DTIM)
BLE	Off
Current Draw	~15-20 mA

Best for

Apps that need persistent Wi-Fi but can tolerate brief radio-off gaps between DTIM beacons.

Enable Modem Sleep

```
#include "esp_wifi.h"

// Min modem sleep — wake at
// every DTIM beacon
esp_wifi_set_ps(
  WIFI_PS_MIN_MODEM
);

// Max modem sleep — longer gap
esp_wifi_set_ps(
  WIFI_PS_MAX_MODEM
);
```

Wi-Fi 6 TWT Advantage

TWT lets the C6 negotiate exact wake slots with the AP, reducing idle RF draw by 10x vs DTIM.

LIGHT SLEEP

Light Sleep Mode

CPU clock is gated and digital peripherals halt, but RAM and register state are fully preserved. Resume is transparent — no reboot.

~130 µA

typical current

Enter Light Sleep

```
#include "esp_sleep.h"

// Wake after 5 seconds
esp_sleep_enable_timer_wakeup(
  5000000ULL
);

esp_light_sleep_start();
// Code resumes here
```

Wake-Up Sources

Timer

RTC timer wakeup

GPIO

Pin level change

Wi-Fi

Incoming packet

UART/I2C

Serial activity

Key advantage: Execution continues exactly where it paused — no driver re-init, no Wi-Fi reconnection overhead.

DEEP SLEEP

Deep Sleep Mode

The most popular low-power mode. The main RISC-V core and HP SRAM are fully powered down. Only the LP core, RTC memory, and selected peripherals remain.

What Stays On?

OFF	Main RISC-V CPU
OFF	HP SRAM
OFF	Wi-Fi / BLE radio
ON	LP Core (ULP)
ON	RTC Memory (16 KB)
ON	RTC Peripherals

~15 µA typical deep sleep draw

Enter Deep Sleep

```
#include "esp_sleep.h"

// Variable persists across sleeps
RTC_DATA_ATTR int bootCount = 0;

void app_main(void) {
  bootCount++;

  // Wake after 30 s
  esp_sleep_enable_timer_wakeup(
    30 * 1000000ULL
  );

  // Enter deep sleep
  esp_deep_sleep_start();
  // never reached
}
```

Note: Wake from deep sleep causes a full reboot. Use RTC_DATA_ATTR or RTC NVS to preserve state.

HIBERNATION

Hibernation & LP Core

The absolute lowest power state — plus the C6's unique LP co-processor for autonomous sensing while the main CPU sleeps.

Deep Sleep vs. Hibernation

Feature	Deep Sleep	Hibernation
Current Draw	~15 µA	~5 µA
LP Core (ULP)	Running	Off
RTC Memory	Preserved (16 KB)	Lost on wake
RTC Peripherals	Maintained	Powered down
Wake Sources	Timer / GPIO / LP	Timer / EXT0 only
Boot on Wake	Full reboot	Full reboot

LP Core Advantage (Deep Sleep only)
The ESP32-C6 LP RISC-V core runs independently — polling sensors without waking the main CPU.

Hibernation Snippet

```
esp_sleep_pd_config(ESP_PD_DOMAIN_RTC_PERIPH, ESP_PD_OPTION_OFF);
esp_deep_sleep_start();
```

Choosing the Right Mode

Match your application's responsiveness and connectivity requirements to the lowest suitable mode.

Continuous computation, streaming, or real-time control loops	**Active**
Persistent Wi-Fi 6 needed; occasional transmission bursts	**Modem Sleep**
Periodic wake required; state must resume without driver re-init	**Light Sleep**
Infrequent wake (seconds-minutes); sensor readings or MQTT push	**Deep Sleep**
Rare wake (hours-days); only RTC timer or GPIO pin needed	**Hibernation**

C6 Exclusive: Wi-Fi 6 TWT can extend battery life dramatically even while staying connected — combine with Modem Sleep.

Summary

Active	80-160 mA	Full RISC-V + Wi-Fi 6 — use when performance is mandatory
Modem Sleep	~15-20 mA	CPU on, radio sleeps between beacons — stays Wi-Fi connected
Light Sleep	~130 µA	CPU gated, state fully preserved — resumes in milliseconds
Deep Sleep	~15 µA	Main CPU/SRAM off, LP core + RTC active — ideal for sensors
Hibernation	~5 µA	Everything off except RTC timer — maximum battery life

Docs: docs.espressif.com/esp32-c6 · github.com/espressif/esp-idf · espressif.com